Decorator

Decorator · Stack compression + encryption on a data store · Build with Waqar

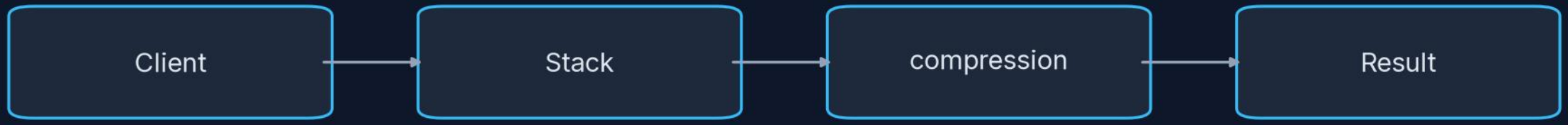
Today's Hook

- Problem: Stack compression + encryption on a data store
- Pattern: Decorator — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Stack compression + encryption on a data store
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Decorator: Stack compression + encryption on a data store

Real-World Scenario

- Running example: Stack compression + encryption on a data store
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```java
package com.buildwithwaqar.patterns.gof.structural.decorator.problem;

import java.util.*;

public final class NaiveStore {
  private String data="";
  public void write(String s){ data="enc("+s+")"; } // one hard-wired transform
  public String read(){ return data; }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```
package com.buildwithwaqar.patterns.gof.structural.decorator.problem;

import java.util.*;

interface DataSource { String read(); void write(String data); }
final class InMemoryDataSource implements DataSource {
  private String data=""; public String read(){return data;} public void write(String d){data=d;}
}
abstract class DataSourceDecorator implements DataSource {
  protected final DataSource wrappee; protected DataSourceDecorator(DataSource w){ wrappee=w; }
  public String read(){ return wrappee.read(); } public void write(String d){ wrappee.write(d); }
}
final class EncryptionDecorator extends DataSourceDecorator {
  public EncryptionDecorator(DataSource w){ super(w); }
  public void write(String d){ wrappee.write("enc("+d+")"); }
  public String read(){ String s=wrappee.read(); return s.startsWith("enc(")? s.substring(4,s.length()-1):s; }
}
final class CompressionDecorator extends DataSourceDecorator {
  public CompressionDecorator(DataSource w){ super(w); }
  public void write(String d){ wrappee.write("zip("+d+")"); }
  public String read(){ String s=wrappee.read(); return s.startsWith("zip(")? s.substring(4,s.length()-1):s; }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.gof.structural.decorator.problem;

import java.util.*;

public final class ProblemDemo {
  public static void main(String[] args) {
    System.out.println("=== Decorator: compress+encrypt store ===");
    System.out.println("--- naive ---");
    var n=new NaiveStore(); n.write("secret"); System.out.println(n.read());
    System.out.println("--- decorator stack ---");
    var mem = new InMemoryDataSource();
    DataSource ds = new EncryptionDecorator(new CompressionDecorator(mem));
    ds.write("secret");
    System.out.println("stored-view=" + mem.read());
    System.out.println("read=" + ds.read());
  }
}
```

Walkthrough Output

```
=== Decorator: compress+encrypt store ===
--- naive ---
enc(secret)
--- decorator stack ---
stored-view=zip(enc(secret))
read=secret
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Decorator.

Related Patterns

- Contrast Decorator with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Decorator → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.